

```

class Father:
    def __init__(self):
        print("Father constructor")

class Mother:
    def __init__(self):
        print("Mother constructor")

class Child(Father, Mother):
    def __init__(self):
        Father.__init__(self)
        Mother.__init__(self)
        print("Child constructor")

Child()

```

```

Father constructor
Mother constructor
Child constructor
<__main__.Child at 0x7e1477078230>

```

```
print(Child.mro())
```

```
[<class '__main__.Child'>, <class '__main__.Father'>, <class '__main__.Mother'>, <class 'object'>]
```

```

class databases:
    def __init__(self):
        print("databases coneect")
        super().__init__()

class login:
    def __init__(self):
        print("login of database")
        super().__init__()

class Application(databases,login):
    def __init__(self):
        print("login of application")
        super().__init__()

```

```
print(Application.mro())
```

```
[<class '__main__.Application'>, <class '__main__.databases'>, <class '__main__.login'>, <class 'object'>]
```

```
A1 = Application()
```

```

login of application
databases coneect
login of database

```

```

class databases:
    def __init__(self):
        super().__init__()
        print("databases coneect")

class login:

```

```
def __init__(self):
    super().__init__()
    print("login of database")

class Application(databases,login):
    def __init__(self):
        super().__init__()
        print("login of application")
```

```
A2 = Application()
```

```
login of database
databases coneect
login of application
```

```
class databases:
    def __init__(self):
        print("databases coneect")

class login:
    def __init__(self):
        print("login of database")

class Application(databases,login):
    def __init__(self):
        super().__init__()
        login.__init__(self)
        print("login of application")
```

```
A2 = Application()
```

```
databases coneect
login of database
login of application
```

```
class databases:
    def __init__(self):
        super().__init__()
        print("databases coneect")

class login:
    def __init__(self):
        print("login of database")

class Application(databases,login):
    def __init__(self):
        super().__init__()
        print("login of application")
```

```
A3 = Application()
```

```
login of database
databases coneect
login of application
```

```
class databases:
    def __init__(self):
        super().__init__()
        print("databases coneect")
```

```
class login:
    def __init__(self):
        super().__init__()
        print("login of database")

class Application(databases,login):
    def __init__(self):
        super().__init__()
        print("login of application")
```

```
A4 = Application()
```

```
login of database
databases conect
login of application
```

```
print(Application.mro())
```

```
[<class '__main__.Application'>, <class '__main__.databases'>, <class '__main__.login'>, <class 'object'>]
```

```
class father:
    def drive(self):
        print("Father can drive")

class mother:
    def cook(self):
        print("Mother can Cook")

class child(father,mother):
    def drive_cook(self):
        print("Child can drive and cook")
```

```
c2 = child()
c2.drive_cook()
c2.cook()
c2.drive()
```

```
Child can drive and cook
Mother can Cook
Father can drive
```

```
class father:
    def drive(self):
        print("Father can drive")

class mother:
    def drive(self):
        print("Mother can drive")

class child(father,mother):
    def drive(self):
        print("Child can drive")
        super().drive()
        mother.drive(self)
```

```
Ch = child()
Ch.drive()
```

```
Child can drive
Father can drive
Mother can drive
```

```
class father:
    def __init__(self):
        print("Father")
        super().__init__()

    def drive(self):
        print("Father can drive")

class mother:
    def __init__(self):
        print("Mother")
        super().__init__()

    def drive(self):
        print("Mother can drive")

class child(father,mother):
    def __init__(self):
        print("child")
        super().__init__()

    def drive(self):
        super().__init__()
        print("Child can drive")

ch1 = child()
```

```
child
Father
Mother
```

```
class father:
    def __init__(self):
        print("Father")
        super().__init__()

    def drive(self):
        print("Father can drive")
        super().drive()

class mother:
    def __init__(self):
        print("Mother")
        super().__init__()

    def drive(self):
        print("Mother can drive")

class child(father,mother):
    def __init__(self):
        print("child")
        super().__init__()

    def drive(self):
```

```
print("Child can drive")
super().drive()
```

```
ch2 = child()
ch2.drive()
```

```
child
Father
Mother
Child can drive
Father can drive
Mother can drive
```

```
c2 = child()
```

```
child
Father
Mother
```

```
c2.drive()
```

```
Child can drive
Father can drive
Mother can drive
```

```
class father:
    def drive(self):
        print("Father can drive")

class mother:
    def drive(self):
        print("Mother can drive")

class child(father,mother):
    pass
```

```
c3 = child()
```

```
c3.drive()
```

```
Father can drive
```

```
class father:
    def drive(self):
        print("Father can drive")

class mother:
    def drive(self):
        print("Mother can drive")

class child(father,mother):
    def drive(self):
        super().__init__()
        print("Child can drive")
```

```
c5 = child()
```

```
c5.drive()
```

Child can drive

```
class father:

    def __init__(self):
        print("Father created")

    def drive(self):
        super().__init__()
        print("Father can drive")

class mother:

    def __init__(self):
        print("Mother created")

    def drive(self):
        super().__init__()
        print("Mother can drive")

class child(father,mother):

    def __init__(self):
        print("child")

    def drive(self):
        super().__init__()
        print("Child can drive")
```

```
c6 = child()
c6.drive()
```

```
child
Father created
Child can drive
```

```
class father:

    def __init__(self):
        super().__init__()
        print("Father created")

    def drive(self):
        print("Father can drive")
        super().drive()

class mother:

    def __init__(self):
        super().__init__()
        print("Mother created")

    def drive(self):
        print("Mother can drive")

class child(father, mother):

    def __init__(self):
        super().__init__()
        print("child")
```

```
def drive(self):
    print("Child can drive")
    super().drive()
```

```
c7 = child()
```

```
Mother created
Father created
child
```

```
c7.drive()
```

```
Child can drive
Father can drive
Mother can drive
```

```
class father:

    def __init__(self):
        super().__init__()
        print("Father created")

    def drive(self):
        print("Father can drive")

class mother:

    def __init__(self):
        super().__init__()
        print("Mother created")

    def drive(self):
        print("Mother can drive")

class child(father, mother):

    def __init__(self):
        super().__init__()
        print("child")

    def drive(self):
        print("Child can drive")
```

```
c9 = child()
```

```
Mother created
Father created
child
```

```
class father:

    def __init__(self):
        print("Father created")
        super().__init__()

    def drive(self):
        print("Father can drive")

class mother:
```

```

def __init__(self):
    print("Mother created")
    super().__init__()

def drive(self):
    print("Mother can drive")

class child(father, mother):

    def __init__(self):
        print("child")
        super().__init__()

    def drive(self):
        print("Child can drive")

```

```
c10 = child()
```

```

child
Father created
Mother created

```

```
c10.drive()
```

```
Child can drive
```

```

class father:

    def __init__(self):
        print("Father created")
        super().__init__()

    def drive(self):
        print("Father can drive")

class mother:

    def __init__(self):
        print("Mother created")
        super().__init__()

    def drive(self):
        print("Mother can drive")

class child(father, mother):

    def __init__(self):
        print("child")
        super().__init__()

    def drive(self):
        super().drive()
        print("Child can drive")

```

```

c11 = child()
c11.drive()

```



```
child
Father created
Mother created
Father can drive
Child can drive
```

```
class father:

    def __init__(self):
        print("Father created")
        super().__init__()

    def drive(self):
        print("Father can drive")
        super().drive()

class mother:

    def __init__(self):
        print("Mother created")
        super().__init__()

    def drive(self):
        print("Mother can drive")

class child(father, mother):

    def __init__(self):
        print("child")
        super().__init__()

    def drive(self):
        print("Child can drive")
        super().drive()
```

```
c12 = child()
print(" ")
c12.drive()
```

```
child
Father created
Mother created

Child can drive
Father can drive
Mother can drive
```

Student Marks (Multiple Inheritance)

✔ Yahan MRO (Method Resolution Order) ka use nahi hua ✔ Kyunki humne explicitly parent constructors call kiye ✔ Isliye order exact wahi hoga jo hum likhenge

```
class internalmarks:

    def __init__(self, imarks):
        self.imarks = imarks
        print("Internal Marks assigned")
```

```

class ExternalMarks:

    def __init__(self,emarks):
        self.emarks = emarks
        print("External Marks Assigned")

class Student(internalmarks,ExternalMarks):

    def __init__(self,imarks,emarks):
        internalmarks.__init__(self,imarks)
        ExternalMarks.__init__(self,emarks)
        print("Student Record Created")

    def total_marks(self):
        return self.imarks + self.emarks

```

```

imarks = int(input("Enter a number : "))
emarks = int(input("Enter a number : "))

s1 = Student(imarks,emarks)
print(s1.total_marks())

```

```

Enter a number : 89
Enter a number : 90
Internal Marks assigned
External Marks Assigned
Student Record Created
179

```

```

class internalmarks:
    def __init__(self, imarks):
        self.imarks = imarks
        print("Internal Marks assigned")

class ExternalMarks:
    def __init__(self, emarks):
        self.emarks = emarks
        print("External Marks Assigned")

class Student(internalmarks, ExternalMarks):
    def __init__(self, imarks, emarks):
        internalmarks.__init__(self, imarks)
        ExternalMarks.__init__(self, emarks)
        print("Student Record Created")

    def total_marks(self):
        return self.imarks + self.emarks

imarks = int(input("Enter internal marks: "))
emarks = int(input("Enter external marks: "))

s1 = Student(imarks, emarks)
print(s1.total_marks())

```

```

Enter internal marks: 123
Enter external marks: 678
Internal Marks assigned
External Marks Assigned
Student Record Created

```

801

```

class internalmarks:

    def __init__(self,imarks):
        self.imarks = 30
        print("Internal Marks assigned")

class ExternalMarks:

    def __init__(self,emarks):
        self.emarks = 40
        print("External Marks Assigned")

class Student(internalmarks,ExternalMarks):

    def __init__(self):
        internalmarks.__init__(self,imarks)
        ExternalMarks.__init__(self,emarks)
        print("Student Record Created")

    def total_marks(self):
        return self.imarks + self.emarks

```

```

class InternalMarks:
    def __init__(self, imarks):
        self.imarks = imarks
        print("Internal Marks assigned")

class ExternalMarks:
    def __init__(self, emarks):
        self.emarks = emarks
        print("External Marks Assigned")

class Student(InternalMarks, ExternalMarks):
    def __init__(self, imarks, emarks):
        InternalMarks.__init__(self, imarks)
        ExternalMarks.__init__(self, emarks)
        print("Student Record Created")

    def total_marks(self):
        return self.imarks + self.emarks

```

```

s1 = Student(50,89)
print(s1.total_marks())

```

```

Internal Marks assigned
External Marks Assigned
Student Record Created
139

```

```

class internalmarks:

    def __init__(self,imarks,*args, **kwargs):
        self.imarks = imarks
        print("internalmraks created")
        super().__init__(*args, **kwargs)

    def get_marks(self):
        return self.imarks

```

```

class ExternalMarks:

    def __init__(self,emarks,*args, **kwargs):
        self.emarks = emarks
        print("ExternalMarks Created")
        super().__init__(*args, **kwargs)

    def get_marks(self):
        return self.emarks

class Students(internalmarks,ExternalMarks):
    def __init__(self,imarks,emarks):
        print("Students Record created")
        super().__init__(imarks,emarks)

    def get_marks(self):
        return self.imarks + self.emarks

```

```

s = Students(78,90)
print(s.get_marks())

```

```

Students Record created
internalmraks created
ExternalMarks Created
168

```

```

class internal:

    def __init__(self,imarks):
        self.imarks = imarks
        print("Internal marks of student ")

    def get_marks(self):
        return self.imarks

class External:

    def __init__(self,emarks):
        self.emarks = emarks
        print("External Marks of students")

    def get_marks(self):
        return self.emarks

class Student_mrks(internal,External):

    def __init__(self,name,imarks,emarks):
        self.name = name
        internal.__init__(self,imarks)
        External.__init__(self,emarks)
        print("Student_mrks Database")

    def total_marks(self):
        return self.imarks + self.emarks

    def get_marks(self):
        return f"total_marks of {self.name} is {self.total_marks()}"

```

```
s1 = Student_mrks("Rahul", 40, 55)

print(s1.get_marks())          # child method
print(f"name of {s1.name} is {internal.get_marks(s1)}")
print(f"name of {s1.name} is {External.get_marks(s1)}")
```

```
Internal marks of student
External Marks of students
Student_mrks Database
total_marks of Rahul is 95
name of Rahul is 40
name of Rahul is 55
```

```
s1 = Student_mrks("Rahul",67,90)
print(s1.total_marks())
print(s1.get_marks())
print(internal.get_marks(s1))
print(External.get_marks(s1))
```

```
Internal marks of student
External Marks of students
Student_mrks Database
157
total_marks of Rahul is 157
67
90
```

```
class internal:

    def __init__(self,imarks,*args,**kwargs):
        self.imarks = imarks
        print("Internal marks of student ")
        super().__init__(*args,**kwargs)

    def get_marks(self):
        return self.imarks

class External:

    def __init__(self,emarks,*args,**kwargs):
        self.emarks = emarks
        print("External Marks of students")
        super().__init__(*args,**kwargs)

    def get_marks(self):
        return self.emarks

class Student_mrks(internal,External):

    def __init__(self,name,imarks,emarks):
        self.name = name
        print("Student_mrks Database")
        super().__init__(imarks,emarks)

    def total_marks(self):
        return self.imarks + self.emarks

    def get_marks(self):
        return f"total_marks of {self.name} is {self.total_marks()}"
```

```
s2 = Student_mrks("Rahul",67,90)
print(s2.total_marks())
print(s2.get_marks())
print(internal.get_marks(s2))
print(External.get_marks(s2))
```

```
Student_mrks Database
Internal marks of student
External Marks of students
157
total_marks of Rahul is 157
67
90
```

```
class internal:

    def __init__(self,imarks,*args,**kwargs):
        self.__imarks = imarks
        print("Internal marks of student ")
        super().__init__(*args,**kwargs)

    def internal_get_marks(self):
        return self.__imarks

class External:

    def __init__(self,emarks,*args,**kwargs):
        self.emarks = emarks
        print("External Marks of students")
        super().__init__(*args,**kwargs)

    def External_get_marks(self):
        return self.emarks

class Student_mrks(internal,External):

    def __init__(self,name,imarks,emarks):
        self.name = name
        print("Student_mrks Database")
        super().__init__(imarks,emarks)

    def total_marks(self):
        return self.internal_get_marks() + self.External_get_marks()

    def get_marks(self):
        return f"total_marks of {self.name} is {self.total_marks()}"
```

```
s2 = Student_mrks("Rahul",67,90)
print(s2.total_marks())
print(s2.get_marks())
```

```
Student_mrks Database
Internal marks of student
External Marks of students
157
total_marks of Rahul is 157
```

```
class Internal:

    def __init__(self, imarks, *args, **kwargs):
        self.__imarks = imarks
        print("Internal marks of student")
```

```

        super().__init__(*args, **kwargs)

    def get_internal_marks(self):
        return self.__imarks

class External:

    def __init__(self, emarks, *args, **kwargs):
        self.emarks = emarks
        print("External marks of students")
        super().__init__(*args, **kwargs)

    def get_external_marks(self):
        return self.emarks

class StudentMarks(Internal, External):

    def __init__(self, name, imarks, emarks):
        self.name = name
        print("StudentMarks Database")
        super().__init__(imarks, emarks) # Corrected

    def total_marks(self):
        # Access internal marks via getter
        return self.get_internal_marks() + self.get_external_marks()

    def get_marks(self):
        return f"Total marks of {self.name} is {self.total_marks()}"

#  Test
s = StudentMarks("Rahul", 45, 50)
print(s.get_marks())

```

```

StudentMarks Database
Internal marks of student
External marks of students
Total marks of Rahul is 95

```